



FINAL PROJECT REPORT

GA-Based Physical Network Topology Optimization Model in MATLAB

TABLE OF CONTENTS

1. Introduction	2
2. Problem Definition and System Model	2
3. GA-Based Network Optimization Approach	2
(A) main.m	3
(B) fitness_network.m	3
(C) visualize_results.m	4
Figure 1: MATLAB implementation of the GA-based network optimization model including the main script and customized fitness function (latest version available on GitHub)	4
4. Results and Technical Interpretation	4
(A) nNode=10	5
(B) nNode=20	5
(C) nNode=30	5
Figure 2: Command window output showing the optimized node coordinates, distance statistics and final fitness value obtained from the genetic algorithm	5
Table 1: Comparative performance of the optimization model for different network sizes	5
(A) nNode=10 (B) nNode=20 (C) nNode=30	6
Figure 3: Convergence plot illustrating the evolution of the best fitness value over generations during the genetic algorithm optimization process (view in GitHub)	6
(A) nNode=10	6
(B) nNode=20	7
(C) nNode=30	7
Figure 4: Optimized network topology and corresponding distance distribution between nodes after the execution of the genetic algorithm (view in GitHub)	7
5. Conclusion	8
6. Acknowledgment	8



1. Introduction

This report is submitted as the final project for the Genetic Algorithms course. It focuses on the application of genetic algorithms to a computer network optimization problem, specifically the physical network topology optimization of multiple nodes. The study is implemented in MATLAB. The project involves the design of a customized fitness function and the use of genetic algorithms to determine optimal node placements under network-related constraints. Program outputs and graphical results are used to evaluate the performance of the proposed optimization model.

2. Problem Definition and System Model

In this project, the problem addressed is the physical topology optimization of a small computer network consisting of five nodes. Each node represents a computer whose position is defined by two-dimensional coordinates (x and y). The objective is to determine optimal node placements that result in an efficient and realistic network topology.

The optimization problem is formulated by considering several practical network-related constraints. Nodes are required to remain within a maximum communication distance (40 units) to preserve connectivity, while a minimum separation distance is enforced to prevent physical overlap. In addition, nodes are assigned different traffic characteristics using a predefined Lookup Table, allowing the optimization process to reflect varying communication demands within the network.

Unlike a fixed-size network model, the proposed system is designed to handle varying numbers of nodes (e.g., 10, 20 and 30 nodes). This introduces additional complexity as larger networks require more sophisticated spatial organization to maintain connectivity and efficiency.

The overall goal is not only to minimize physical distances between nodes but also to incorporate network behavior such as traffic-aware communication cost and mandatory routing requirements. This formulation allows the genetic algorithm to produce network layouts that are both physically feasible and meaningful from a computer network perspective while also enabling scalability analysis.

3. GA-Based Network Optimization Approach

A genetic algorithm is used to solve the defined network topology optimization problem. Each individual in the population represents a candidate network configuration encoded as a vector containing the x and y coordinates of all network nodes. The genetic algorithm iteratively improves these configurations by minimizing a customized fitness function.

The fitness function evaluates each candidate solution based on three main components; traffic-weighted communication cost, connectivity constraints and node separation constraints. Communication cost is calculated by weighting inter-node distances according to node traffic levels. Connectivity is enforced by penalizing links that exceed the maximum allowable distance while node overlap is prevented through a minimum separation penalty. Mandatory routing constraints are also included to ensure that specific node connections are always maintained.



```

main.m x fitness_network.m x visualize_results.m x +
1  %% Network Topology Optimization using Genetic Algorithm
2  % Goal: Optimize the physical placement of 5 computers (nodes) in a 2D space using a genetic algorithm.
3  % Decision variables: x and y coordinates of each node
4  % Overall idea: GA searches for coordinate configurations that minimize communication cost while satisfying network constraints.
5  clc; clear; close all;
6
7  %% Output directory for saving figures
8  outputDir = './outputs';
9
10 if ~exist(outputDir, 'dir')
11     mkdir(outputDir);
12 end
13
14 %% Problem Parameters
15 nNodes = 5;           % Number of network nodes
16 nvars = nNodes * 2;   % Total variables: [x1 y1 x2 y2 ...]
17 lb = zeros(1, nvars); % Lower bounds (0)
18 ub = ones(1, nvars) * 100; % Upper bounds (100)
19
20 %% Genetic Algorithm Configuration
21 options = gaoptimset(...
22     'PopulationSize', 50, ... % Number of individuals
23     'Generations', 200, ... % Number of iterations
24     'Display', 'iter', ... % Show progress in command window
25     'PlotFcn', @gaplotbestf); % Plot best fitness per generation
26
27 %% Run Genetic Algorithm
28 % Calls custom fitness function to evaluate solutions
29 fitnessFcn = @fitness_network;
30 [x_best, fval] = ga(fitnessFcn, nvars, [], [], [], [], lb, ub, [], options);
31
32 %% Save convergence plot (fitness vs generations)
33 figure(1);
34 title('GA Convergence Plot - Best Fitness per Generation');
35 exportgraphics(gcf, ...
36     fullfile(outputDir, 'convergence_plot.png'), ...
37     'Resolution', 300);
38
39 %% Visualize results and print outputs
40 visualize_results(x_best, fval, nNodes, outputDir);

```

(A) main.m

```

main.m x fitness_network.m x visualize_results.m x +
1  %% Fitness Function
2  % Network Topology Optimization Fitness Function
3  % Input: x = [x1, y1, x2, y2, ..., xn, yn] (node coordinates)
4  % Output: score = fitness value (to be minimized)
5  % Objective:
6  % - Minimize traffic-weighted communication cost
7  % - Enforce connectivity constraints (max distance)
8  % - Prevent node overlap (min separation)
9  % - Maintain mandatory routing links
10 function score = fitness_network(x)
11 % Convert vector into coordinate matrix [x y]
12 coords = reshape(x, [], 2);
13 n = size(coords,1);
14
15 %% Lookup Table (Node properties)
16 % Column 1: traffic load (importance of node)
17 % Column 2: jitter sensitivity (not directly used but included for realism)
18 nodeProps = [
19     5 1.2;
20     3 1.0;
21     4 1.1;
22     2 0.8;
23     1 0.6
24 ];
25
26 %% Mandatory routing links (must remain connected)
27 mandatoryLinks = [
28     1 2;
29     1 3;
30     1 4;
31     1 5
32 ];
33
34 %% Distance constraints
35 maxDist = 40; % Maximum allowed communication distance
36 minSeparation = 5; % Minimum distance to prevent overlap
37
38 %% Initialize cost terms
39 totalCost = 0; % Communication cost
40 connectivityPenalty = 0; % Penalty for long links
41 overlapPenalty = 0; % Penalty for overlapping nodes
42
43 %% Evaluate all node pairs
44 for i = 1:n
45     for j = i+1:n
46         d = norm(coords(i,:) - coords(j,:)); % Euclidean distance
47
48         % Traffic-aware cost (important nodes get higher weight)
49         trafficWeight = nodeProps(i,1) + nodeProps(j,1);
50         totalCost = totalCost + trafficWeight * d;
51
52         % Penalty if nodes are too far apart (breaks connectivity)
53         if d > maxDist
54             connectivityPenalty = connectivityPenalty + 200;
55         end
56
57         % Penalty if nodes are too close (physical overlap)
58         if d < minSeparation
59             overlapPenalty = overlapPenalty + 1000;
60         end
61     end
62 end
63
64 %% Enforce mandatory routing constraints
65 for k = 1:size(mandatoryLinks,1)
66     i = mandatoryLinks(k,1);
67     j = mandatoryLinks(k,2);
68     d = norm(coords(i,:) - coords(j,:));
69
70     % Additional penalty if mandatory link is broken
71     if d > maxDist
72         connectivityPenalty = connectivityPenalty + 500;
73     end
74 end
75
76 %% Final fitness score (lower is better)
77 score = totalCost + connectivityPenalty + overlapPenalty;
78 end

```

(B) fitness_network.m

```

main.m x fitness_network.m x visualize_results.m x +
1 function visualize_results(x_best, fval, nNodes, outputDir)
2
3 % Convert optimized solution to coordinates
4 coords = reshape(x_best, [], 2);
5
6 % Create figure window
7 fig = figure('Position', [100, 100, 1200, 500]);
8
9 % Graph 1: Optimized Network Topology
10 subplot(1, 2, 1);
11
12 % Plot node positions
13 scatter(coords(:,1), coords(:,2), 200, 'filled', 'MarkerFaceColor', 'b');
14 hold on;
15
16 % Label nodes
17 for i = 1:nNodes
18     text(coords(i,1)+2, coords(i,2)+2, ...
19         ['Node ' num2str(i)], 'FontSize', 10);
20 end
21
22 title('Optimized Network Topology', 'FontSize', 12, 'FontWeight', 'bold');
23 xlabel('X Coordinate');
24 ylabel('Y Coordinate');
25 grid on;
26 axis([0 100 0 100]);
27
28 % Draw connections between nodes within range
29 for i = 1:nNodes
30     for j = i+1:nNodes
31         d = norm(coords(i,:) - coords(j,:));
32
33         if d <= 40 % Only valid communication links
34             plot([coords(i,1), coords(j,1)], ...
35                 [coords(i,2), coords(j,2)], ...
36                 'k--', 'LineWidth', 1.5);
37         end
38     end
39 end
40
41 % Graph 2: Distance Distribution
42 subplot(1, 2, 2);
43
44 distances = [];
45
46 % Compute all pairwise distances
47 for i = 1:nNodes
48     for j = i+1:nNodes
49         distances = [distances, norm(coords(i,:) - coords(j,:))];
50     end
51 end
52
53 % Plot histogram of distances
54 histogram(distances, 10, 'FaceColor', 'cyan');
55 title('Distance Distribution', 'FontSize', 12, 'FontWeight', 'bold');
56 xlabel('Distance (unit)');
57 ylabel('Frequency');
58 hold on;
59
60 % Show connectivity threshold
61 xline(40, 'r--', 'LineWidth', 2, ...
62     'Label', 'Max Connectivity (40)');
63
64 % Save figure
65 exportgraphics(fig, ...
66     fullfile(outputDir, 'network_topology_results.png'), ...
67     'Resolution', 300);
68
69 % Print results to command window
70 fprintf('\n===== \n');
71 fprintf('NETWORK TOPOLOGY OPTIMIZATION \n');
72 fprintf('===== \n');
73
74 fprintf('Optimal Coordinates: \n');
75 for i = 1:nNodes
76     fprintf('Node %d: (%.2f, %.2f) \n', ...
77         i, coords(i,1), coords(i,2));
78 end
79
80 fprintf('\nDistance Summary: \n');
81 fprintf('Total Distance: %.2f \n', sum(distances));
82 fprintf('Average Distance: %.2f \n', mean(distances));
83 fprintf('Min Distance: %.2f \n', min(distances));
84 fprintf('Max Distance: %.2f \n', max(distances));
85
86 fprintf('\nFitness Value (Best): %.2f \n', fval);
87
88 % Count connections exceeding threshold
89 fprintf('Connectivity Penalty: ');
90 penalty_count = sum(distances > 40);
91 fprintf('%d (40 units and above %d connection) \n', ...
92     penalty_count*100, penalty_count);
93
94 fprintf('\n===== \n');
95 end

```

(C) visualize_results.m

Figure 1: MATLAB implementation of the GA-based network optimization model including the main script and customized fitness function (latest version available on [GitHub](#))

MATLAB's built-in genetic algorithm solver (*ga*) is employed for optimization while the problem-specific behavior is fully controlled through the customized fitness function. [Figure 1](#) presents the main MATLAB code structure including the genetic algorithm configuration and fitness evaluation process.

The model was extended to support scalable node configurations requiring dynamic handling of node-specific parameters such as traffic properties and routing constraints. This allows the same optimization framework to be applied to networks of increasing size without structural changes to the algorithm.

4. Results and Technical Interpretation

The optimization process produces network topologies whose structure varies significantly depending on the number of nodes. The command window outputs ([Figure 2](#)) show the optimized node coordinates, distance statistics and final fitness values for different network sizes.

```
=====
NETWORK TOPOLOGY OPTIMIZATION
=====

Optimal Coordinates:
Node 1: (64.26, 63.49)
Node 2: (41.88, 55.57)
Node 3: (42.82, 61.94)
Node 4: (35.56, 70.85)
Node 5: (66.30, 56.79)
Node 6: (75.67, 55.32)
Node 7: (59.15, 49.20)
Node 8: (48.91, 65.26)
Node 9: (54.70, 59.96)
Node 10: (69.08, 51.40)

Distance Summary:
Total Distance: 846.03
Average Distance: 18.80
Min Distance: 6.07
Max Distance: 43.01

Fitness Value (Best): 3399.06
Connectivity Penalty: 100 (40 units and above 1 connection)
=====
```

(A) nNode=10

```
Command Window
=====
NETWORK TOPOLOGY OPTIMIZATION
=====

Optimal Coordinates:
Node 1: (50.81, 39.66)
Node 2: (38.37, 60.91)
Node 3: (47.89, 59.38)
Node 4: (47.00, 33.77)
Node 5: (50.75, 65.85)
Node 6: (44.58, 75.31)
Node 7: (35.07, 33.16)
Node 8: (68.74, 25.88)
Node 9: (59.08, 48.40)
Node 10: (44.64, 66.69)
Node 11: (44.69, 48.52)
Node 12: (53.56, 54.73)
Node 13: (40.42, 45.04)
Node 14: (31.81, 40.93)
Node 15: (58.82, 40.64)
Node 16: (50.82, 46.01)
Node 17: (69.83, 40.22)
Node 18: (29.56, 61.41)
Node 19: (41.45, 37.74)
Node 20: (54.45, 54.76)

Distance Summary:
Total Distance: 4110.09
Average Distance: 21.63
Min Distance: 0.90
Max Distance: 55.02

Fitness Value (Best): 19387.76
Connectivity Penalty: 900 (40 units and above 9 connection)
=====
```

(B) nNode=20

```
Command Window
=====
NETWORK TOPOLOGY OPTIMIZATION
=====

Optimal Coordinates:
Node 1: (33.83, 40.54)
Node 2: (67.37, 40.39)
Node 3: (41.29, 66.15)
Node 4: (42.35, 41.15)
Node 5: (29.73, 33.55)
Node 6: (29.49, 51.90)
Node 7: (22.27, 60.73)
Node 8: (82.90, 73.08)
Node 9: (51.89, 53.73)
Node 10: (39.31, 45.19)
Node 11: (45.74, 63.65)
Node 12: (52.33, 70.04)
Node 13: (24.21, 70.59)
Node 14: (50.78, 60.31)
Node 15: (46.00, 48.33)
Node 16: (66.87, 49.01)
Node 17: (8.86, 55.97)
Node 18: (46.55, 68.63)
Node 19: (59.70, 55.09)
Node 20: (23.90, 43.81)
Node 21: (32.63, 53.16)
Node 22: (37.51, 49.90)
Node 23: (46.46, 54.59)
Node 24: (37.09, 36.60)
Node 25: (51.62, 21.71)
Node 26: (27.39, 47.09)
Node 27: (13.49, 71.37)
Node 28: (36.42, 26.34)
Node 29: (34.30, 69.51)
Node 30: (33.85, 45.90)

Distance Summary:
Total Distance: 11472.68
Average Distance: 26.37
Min Distance: 3.38
Max Distance: 75.99

Fitness Value (Best): 66997.52
Connectivity Penalty: 6000 (40 units and above 60 connection)
=====
```

(C) nNode=30

Figure 2: Command window output showing the optimized node coordinates, distance statistics and final fitness value obtained from the genetic algorithm

The results for different node sizes are summarized in [Table 1](#). For nNode = 10, the network remains compact with most distances well below the connectivity threshold and only minor constraint violations observed. The algorithm successfully balances communication cost and connectivity requirements.

For nNode = 20, the network begins to spread slightly resulting in increased distances and moderate connectivity penalties. Although most nodes remain clustered, several nodes move further apart indicating the beginning of scalability limitations.

For nNode = 30, the network structure changes significantly. While a central cluster still exists, several nodes are positioned far from the main cluster resulting in long communication links and a high number of connectivity violations. This indicates that a single central gateway structure is no longer sufficient for maintaining efficient communication in large networks.

Table 1: Comparative performance of the optimization model for different network sizes

Nodes	Avg Distance	Max Distance	Penalty	Behavior
10	~19	~ 43	Low	Compact and stable
20	~ 23	~ 56	Medium	Partially stretched
30	~ 26	~ 76	High	Constraint violations

The convergence behavior of the genetic algorithm is illustrated in [Figure 3](#). Across all cases, the best fitness value decreases rapidly during early generations and stabilizes over time indicating effective optimization. However, for larger networks, the final fitness values remain significantly higher due to increased penalties and structural complexity.

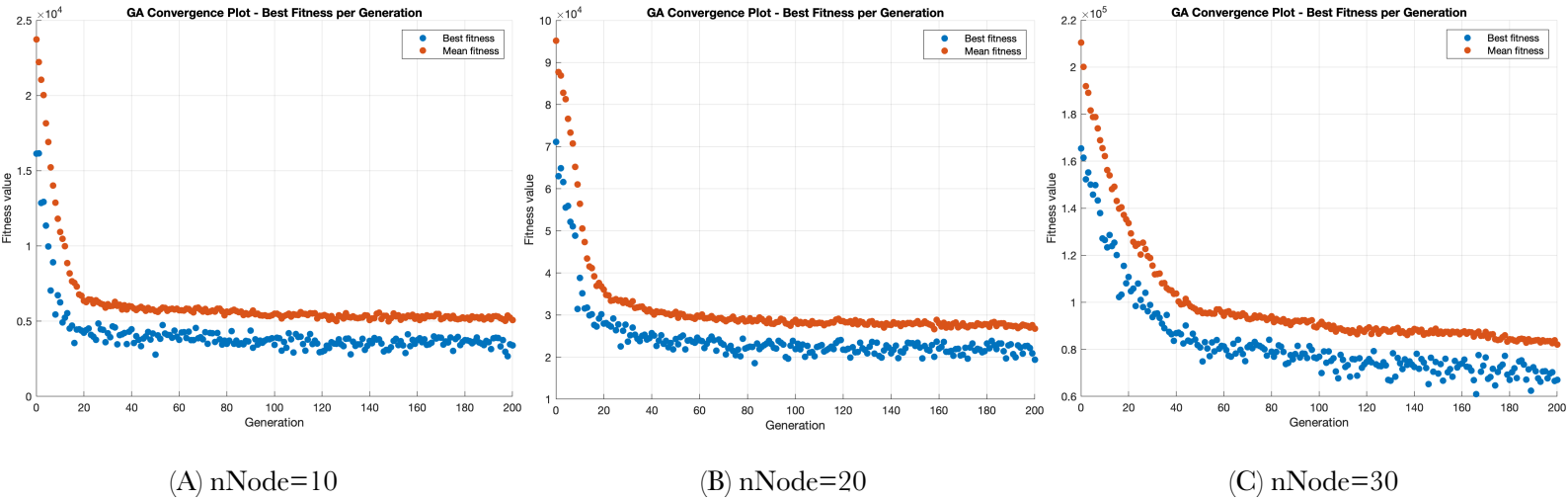
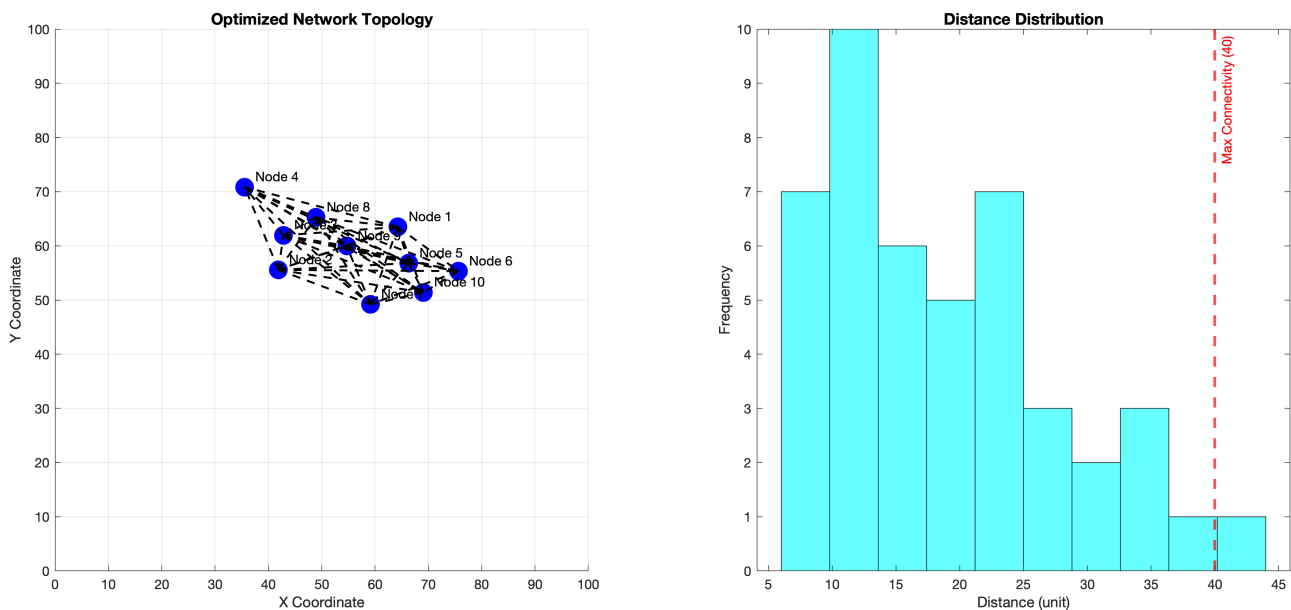
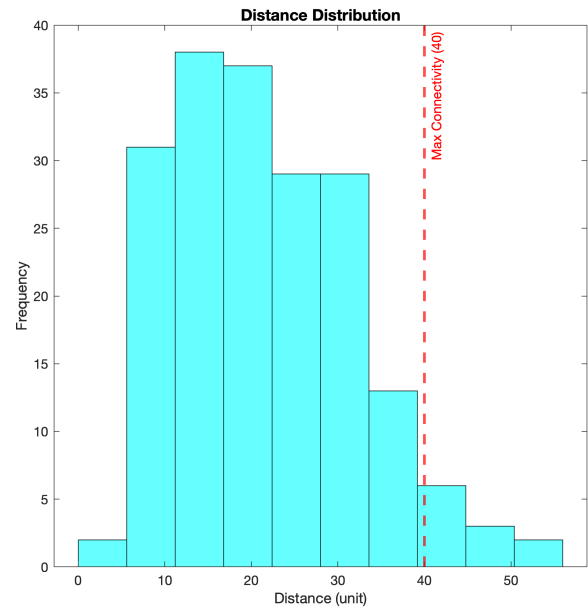
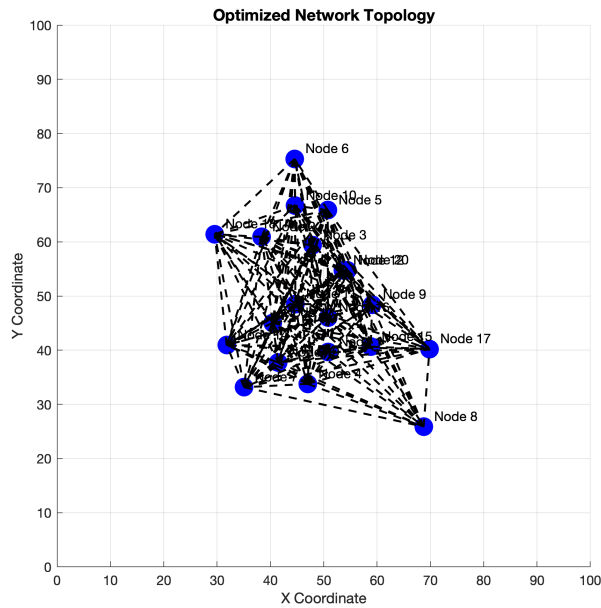


Figure 3: Convergence plot illustrating the evolution of the best fitness value over generations during the genetic algorithm optimization process ([view in GitHub](#))

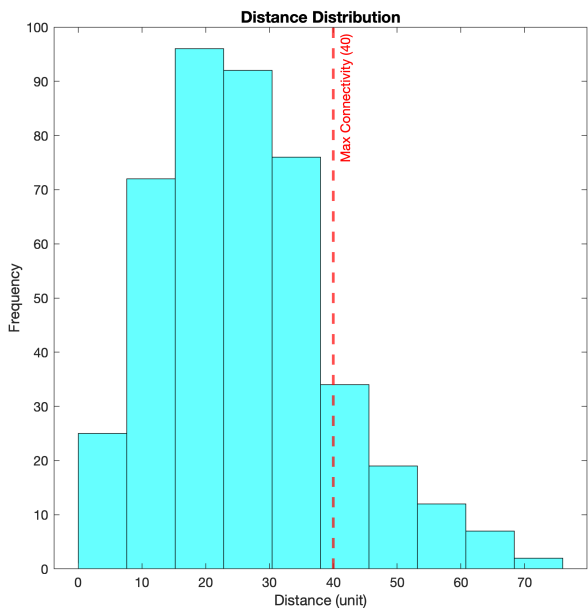
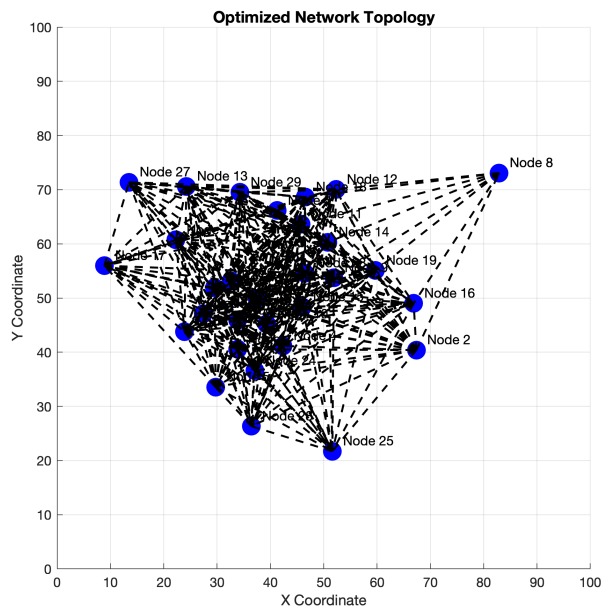
[Figure 4](#) shows the final optimized network topology and the corresponding distance distribution. For smaller networks, the topology exhibits a well-balanced and compact structure with distances concentrated within a narrow range. As the number of nodes increases, the distance distribution becomes wider and a greater number of connections exceed the maximum allowable distance. This results in increased penalties and reduced network efficiency.



(A) nNode=10



(B) nNode=20



(C) nNode=30

Figure 4: Optimized network topology and corresponding distance distribution between nodes after the execution of the genetic algorithm ([view in GitHub](#))

The results clearly demonstrate that while the proposed model performs effectively for small and medium-sized networks, its performance degrades for larger networks due to increasing connectivity violations. This behavior highlights a fundamental limitation of single-gateway network structures.



5. Conclusion

In this final project, the application of genetic algorithms to a computer network optimization problem was studied and implemented using MATLAB. The focus of the project was the physical network topology optimization of multiple nodes under realistic constraints.

A customized fitness function was developed to evaluate node placements by considering traffic-aware communication cost, connectivity limits, mandatory routing requirements and minimum node separation. The results obtained from the genetic algorithm demonstrate that the optimization model produces feasible and efficient network topologies for small and medium-sized networks.

The study further shows that as the number of nodes increases, the optimization problem becomes more complex leading to increased connectivity violations and less efficient network structures. This indicates that a single-gateway topology is not sufficient for large-scale networks.

Overall, this project highlights both the strengths and limitations of genetic algorithm-based optimization in network design. While effective for moderate network sizes, larger systems require more advanced approaches.

Future work may include the integration of multi-gateway structures or clustered network designs to improve scalability and performance in large-scale networks.

6. Acknowledgment

The author used Microsoft Copilot as a writing and organization aid. All technical design, implementation, experiments and scientific analysis were carried out by the author and full human judgment was maintained throughout the project.

The complete source code and experimental results are available in the project's [GitHub repository](#) released under the MIT License.